BuildNDeploy

AWS Three-Tier Web Application with Automated CI/CD

Project Documentation By: Sumit Milind Bhamkar

Project Type: Cloud Computing and DevOps
Cloud Provider: Amazon Web Services (AWS)
Region: US East (N. Virginia) — us-east-1
Domain: buildndeploy.online
Source Code: https://github.com/Sumit-bhamkar/buildndeploy
Deployment Approach: Docker on Amazon EC2 with Load Balancing and Auto Scaling

---

1. Project Overview

BuildNDeploy is a cloud-based web application deployment project built to gain hands-on experience with AWS infrastructure, Docker, networking, security, and automated CI/CD.

The project uses a three-tier architecture with separate frontend, backend, and database layers. The frontend is built using React and served through Nginx, the backend uses Node.js and Express.js, and application data is stored in Amazon RDS for MySQL.

The infrastructure includes a custom VPC, public and private subnets, two Application Load Balancers, EC2 Auto Scaling Groups, Amazon ECR, and AWS Systems Manager Parameter Store. GitHub Actions automates the application build and deployment workflow.

The main objective was to understand how individual AWS services work together to deploy, update, and manage a containerized web application.

Technologies Used

- **Cloud:** Amazon Web Services (AWS)
- **Frontend:** React, Vite, Nginx
- **Backend:** Node.js, Express.js
- **Database:** Amazon RDS for MySQL
- **Containerization:** Docker
- **Container Registry:** Amazon ECR
- **Compute:** Amazon EC2
- **Load Balancing:** Application Load Balancer

- **Scaling:** EC2 Auto Scaling Groups
- **Networking:** VPC, subnets, route tables, Internet Gateway, NAT Gateways, Security Groups, NACLs
- **Configuration and Secrets:** AWS Systems Manager Parameter Store
- **DNS:** Amazon Route 53
- **CI/CD:** GitHub Actions
- **Authentication:** AWS IAM and GitHub OIDC
- **Version Control:** Git and GitHub

2. Architecture Overview

BuildNDeploy was designed using a three-tier architecture distributed across two Availability Zones in the us-east-1 Region.

The frontend and backend run on separate EC2 Auto Scaling Groups, while Amazon RDS provides managed database hosting. A public Application Load Balancer receives external HTTP traffic, and an internal Application Load Balancer distributes requests to backend instances.

The VPC separates internet-facing infrastructure from private application resources. NAT Gateways provide outbound internet connectivity for eligible private resources, while Security Groups control communication between the different components.

2.1 Architecture Diagram

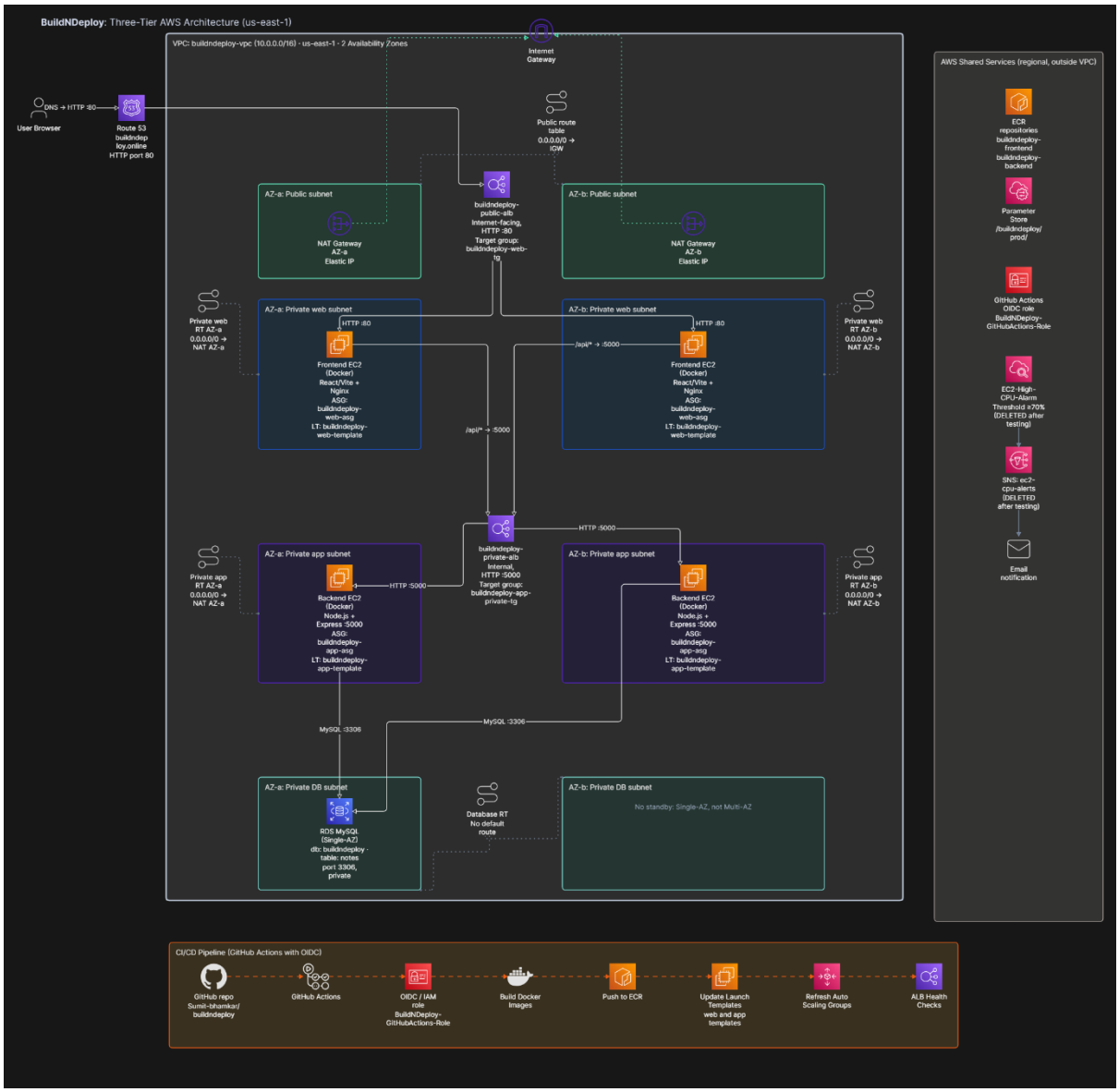


Figure 1: Overall AWS architecture showing the VPC, Availability Zones, public and private subnets, load balancers, EC2 instances, Auto Scaling Groups, RDS, NAT Gateways, and application traffic flow.

2.2 High-Level Architecture Flow

External Request Flow

User Browser → Amazon Route 53 → buildndeploy-public-alb → buildndeploy-web-tg → Frontend EC2 Instances → React Application

Backend API Flow

React Application → Nginx API Proxy (/api/) → buildndeploy-private-alb → buildndeploy-app-private-tg → Backend EC2 Instances → Amazon RDS for MySQL

Deployment Flow

GitHub Push → GitHub Actions → Docker Build → Amazon ECR → Launch Template Update → Auto Scaling Instance Refresh → Updated Application

The internal ALB is not directly accessible from the public internet. The frontend Nginx configuration routes API requests to the internal ALB, allowing the browser to communicate through the frontend endpoint.

3. AWS Networking Architecture

3.1 Amazon VPC

The project uses a custom VPC named buildndeploy-vpc.

The VPC provides an isolated networking environment for the application infrastructure. Its subnets, route tables, gateways, and security rules determine how the frontend, backend, database, and load balancers communicate.

The network was designed to separate public-facing entry points from private application resources.

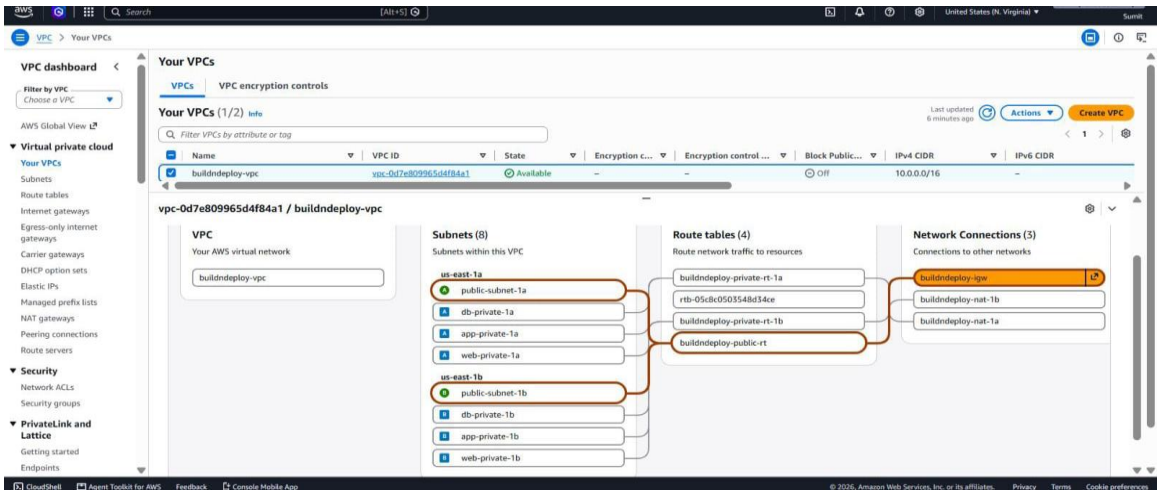


Figure 2: Custom VPC created for the BuildNDeploy project.

3.2 Availability Zones and Subnets

The architecture uses two Availability Zones to distribute application resources.

The subnet naming convention used in the project includes:

Subnet Name	Purpose

public-subnet-1a	Public-facing infrastructure in the first AZ
public-subnet-1b	Public-facing infrastructure in the second AZ
app-private-1a	Private application resources in the first AZ
app-private-1b	Private application resources in the second AZ
db-private-1a	Private database subnet in the first AZ
db-private-1b	Private database subnet in the second AZ

The frontend and backend compute resources are kept separate from the public entry point. The database is placed in the private database layer.

A DB subnet group spanning two Availability Zones does not mean the database itself is deployed in Multi-AZ mode.

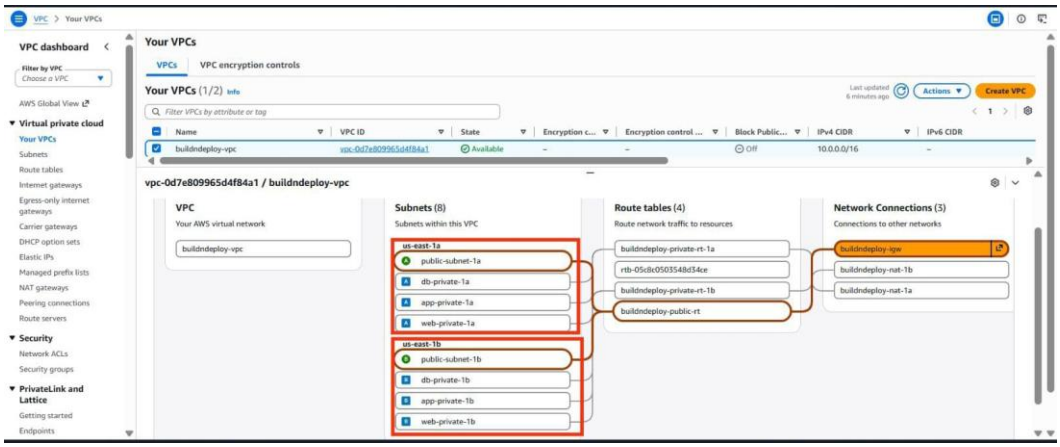


Figure 3: Subnet organization across the two Availability Zones.

3.3 Internet Gateway

The Internet Gateway is attached to buildndeploy-vpc to provide internet connectivity for appropriately routed public resources.

The public route table directs eligible internet-bound IPv4 traffic through the Internet Gateway.

The Internet Gateway does not automatically make every resource in the VPC publicly accessible. Public IP addressing, routing, Security Groups, and service configuration also determine accessibility.

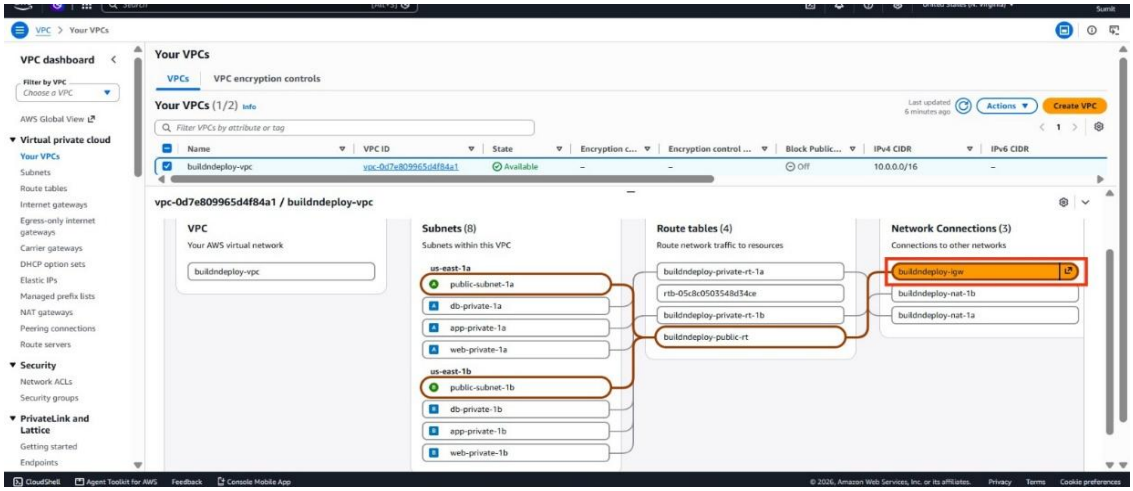


Figure 4: Internet Gateway attached to the project VPC.

3.4 NAT Gateways

Two NAT Gateways were configured in the architecture because the application uses two Availability Zones (AZs). Each NAT Gateway is placed in a public subnet in a different AZ, allowing private subnets to access the internet for outbound traffic.

This design improves availability and reduces cross-AZ traffic by allowing each AZ to use its own NAT Gateway. If one AZ experiences an issue, the other AZ can continue operating independently, provided its resources and network paths remain healthy.

Private instances may need outbound internet access to download packages, retrieve updates, or access external services without being directly exposed to inbound internet traffic.

Traffic Flow:

Private EC2 (AZ 1) → Private Route Table → NAT Gateway (AZ 1) → Internet Gateway → Internet

Private EC2 (AZ 2) → Private Route Table → NAT Gateway (AZ 2) → Internet Gateway → Internet

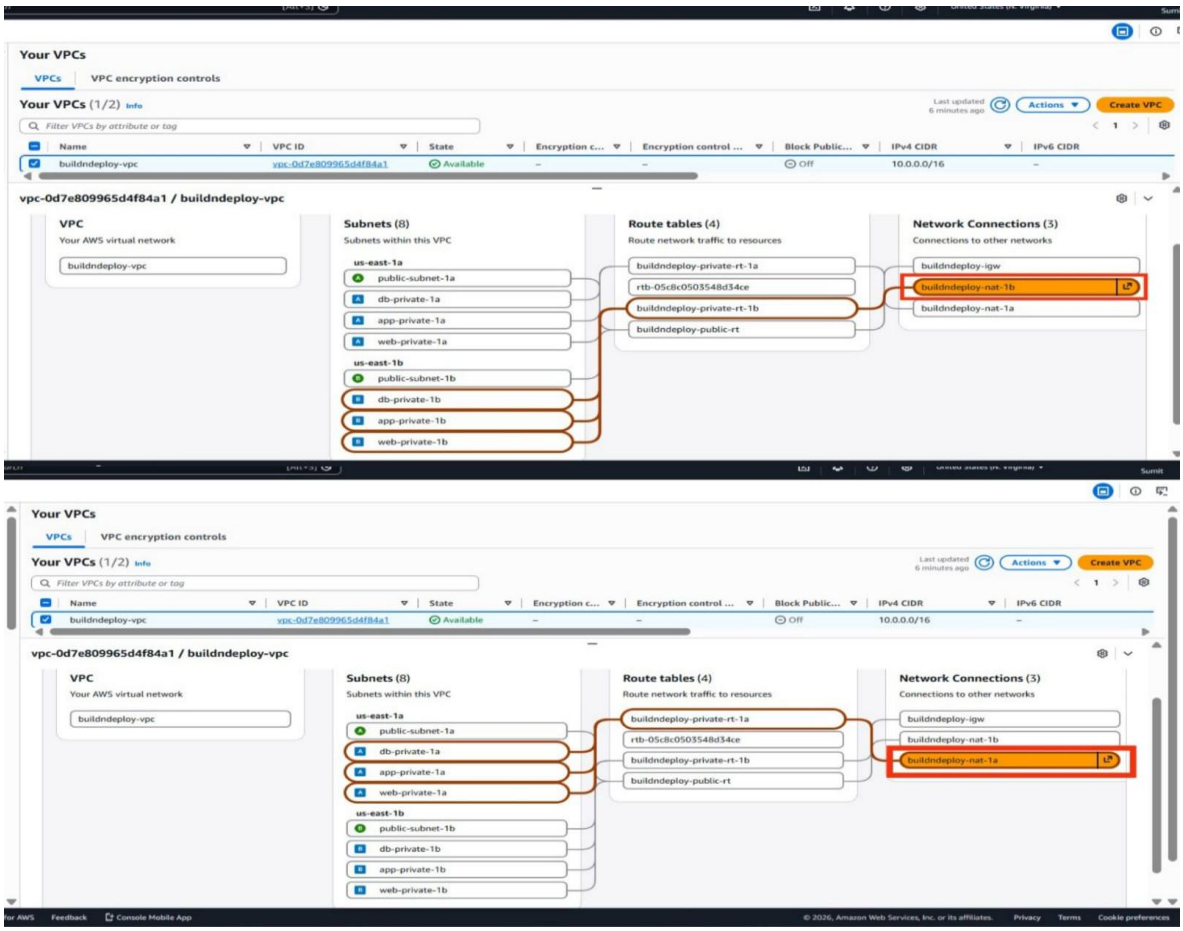


Figure 5: Two NAT Gateways deployed across separate Availability Zones to support outbound internet access and improve network availability.

3.5 Route Tables

Route tables control how network traffic is routed between the VPC and other destinations.

The public route table uses the Internet Gateway for eligible internet-bound traffic. Private route tables use the appropriate NAT Gateway where outbound internet connectivity is required.

Local VPC routes allow communication between resources when the applicable network and security rules permit it.

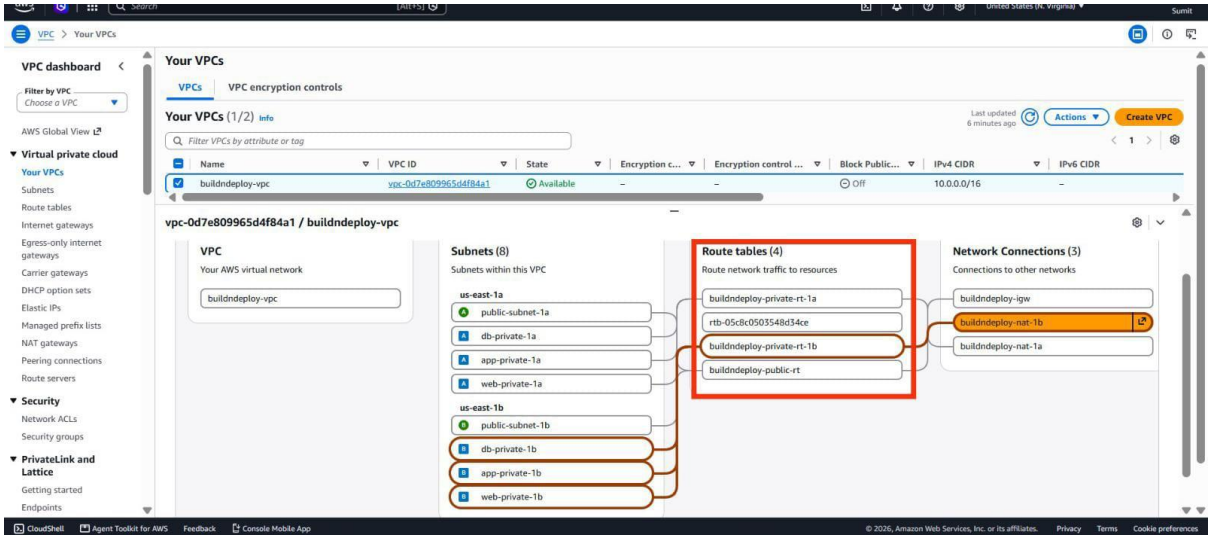


Figure 6: Route table configuration showing the relevant network paths.

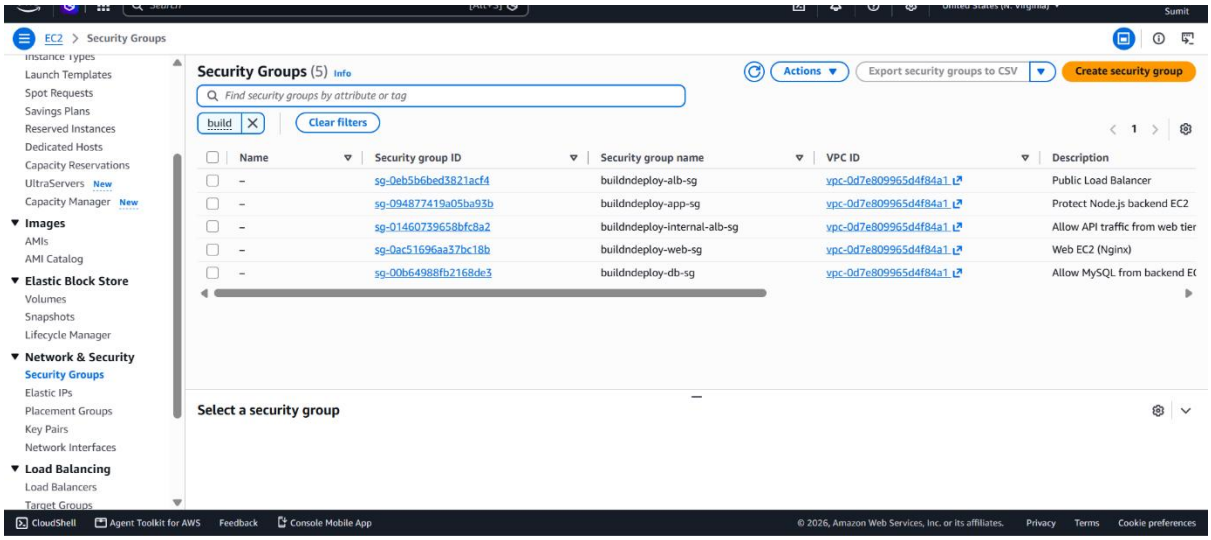
3.6 Security Groups and Network ACLs

Security Groups act as stateful firewalls associated with supported AWS resources. Network ACLs provide additional stateless controls at the subnet level.

The intended communication pattern is:

- Public ALB: accepts permitted HTTP traffic from external users.
- Frontend EC2: accepts application traffic from the public ALB.
- Internal ALB: accepts API traffic from the configured frontend routing layer.
- Backend EC2: accepts application traffic from the internal ALB.
- RDS: accepts MySQL connections from the authorized backend security group.

This design limits direct access to private resources and separates external traffic from internal application communication.



Sumit Milind Bhamkar || LinkedIn || GitHub

Figure 7: Security Group configurations controlling inbound and outbound traffic between the public load balancer, frontend, backend, and database layers. Redact account identifiers and any sensitive information before publication.

4. Public Application Load Balancer

The public Application Load Balancer is named buildndeploy-public-alb.

It acts as the entry point for HTTP traffic coming from users. It is configured as an internet-facing load balancer and deployed in public subnets.

When a user visits the application domain, the request reaches the public ALB, which forwards it to a healthy target registered in the frontend target group.

Traffic Flow

User → buildndeploy-public-alb → buildndeploy-web-tg → Frontend EC2

The target group allows the load balancer to identify registered frontend targets and perform health checks.

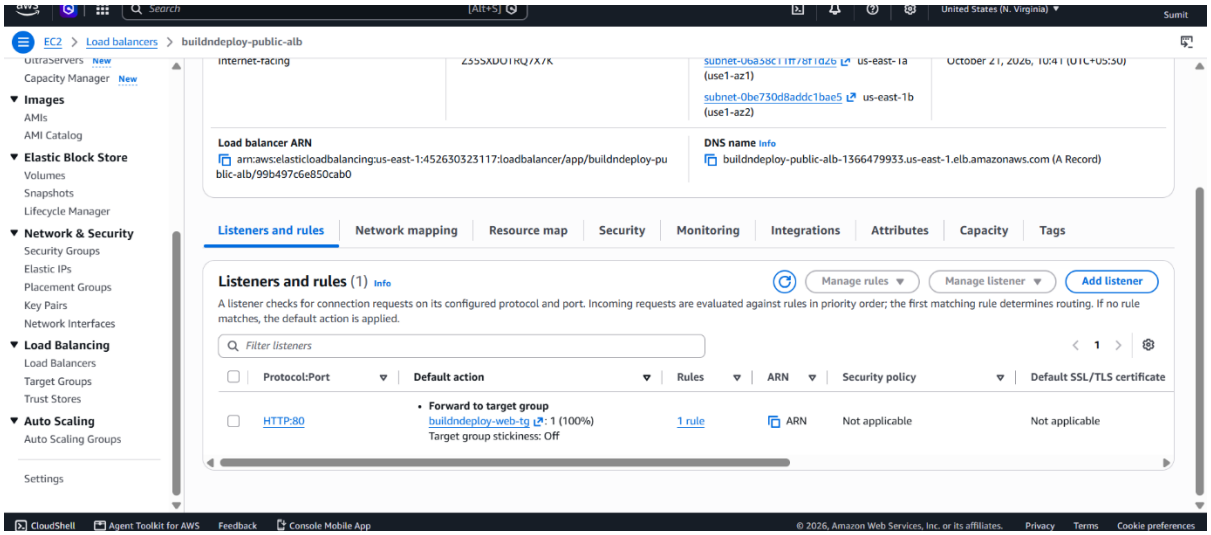


Figure 8: Public Application Load Balancer configuration.

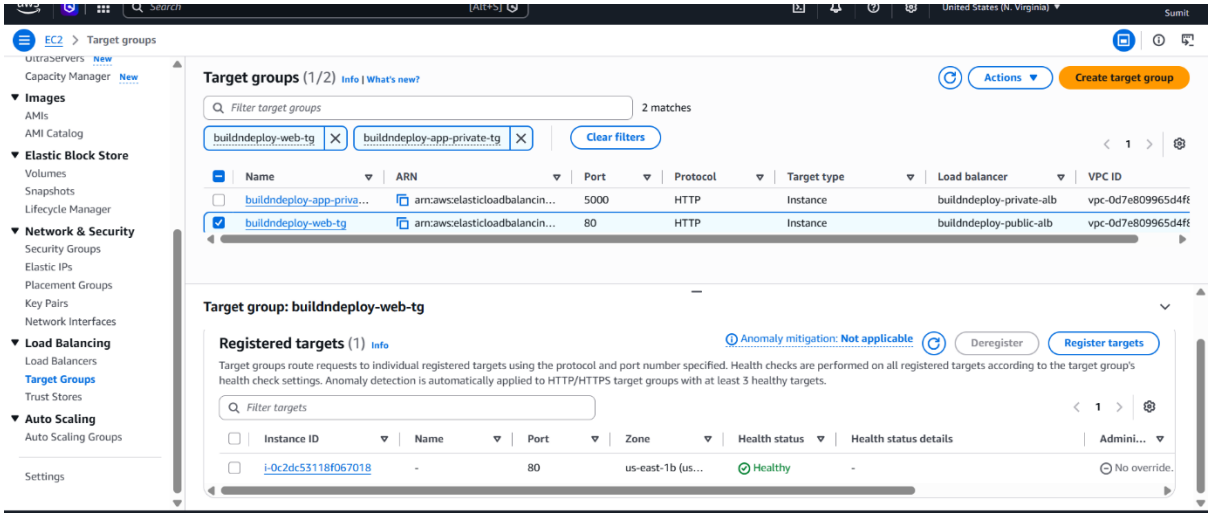


Figure 9: Frontend target group and registered target health.

5. Frontend Layer

The frontend runs on EC2 instances managed by an Auto Scaling Group named buildndeploy-web-asg.

The frontend uses React and Vite, while Nginx serves the built frontend files from a Docker container. Nginx also handles requests beginning with /api/ and forwards them to the internal backend load balancer according to the configured proxy rules.

The frontend EC2 instances are located in private web subnets. They receive application traffic through the public ALB rather than requiring direct public access.

Frontend Components

- React and Vite for the user interface.
- Nginx for serving frontend assets and proxying API requests.
- Docker for packaging the frontend application.
- EC2 for running the frontend container.
- buildndeploy-web-asg for managing frontend instance capacity.
- buildndeploy-web-template for defining frontend instance launch configuration.

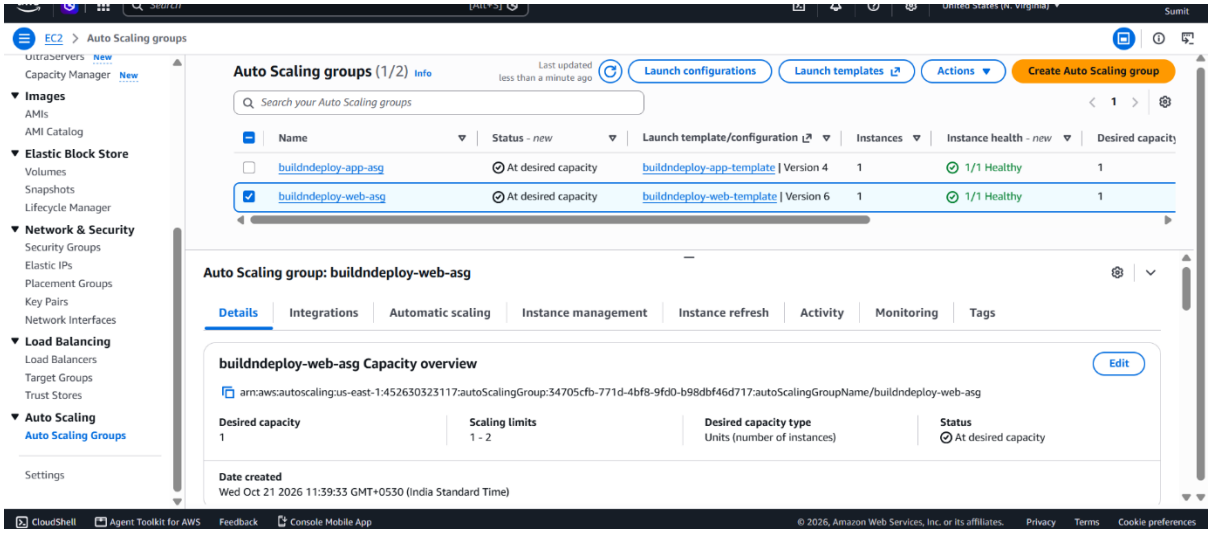


Figure 10: Frontend Auto Scaling Group configuration.

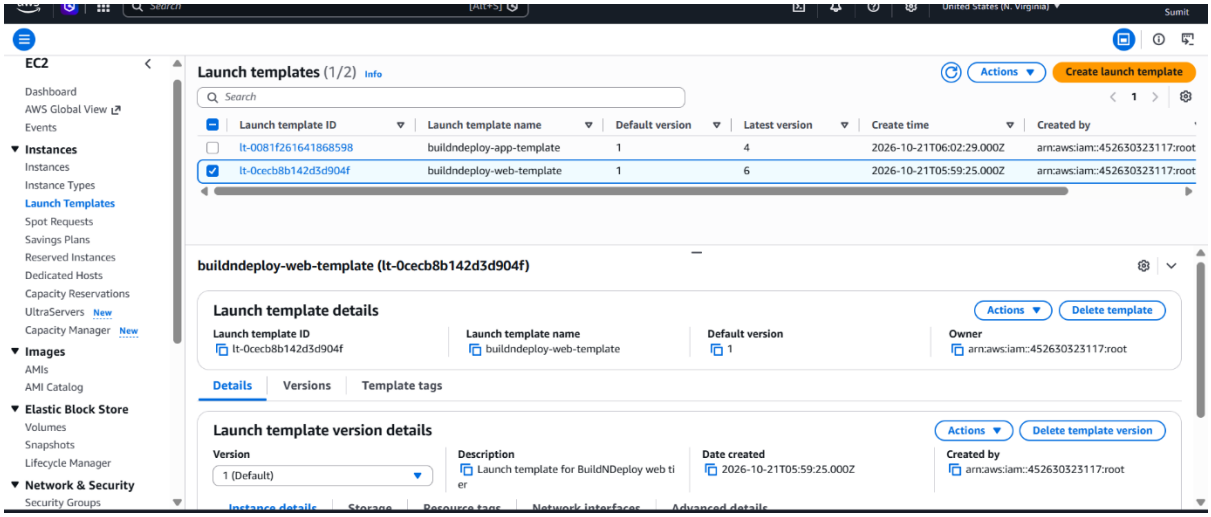


Figure 11: Launch Template used for frontend EC2 instances.

6. Internal Application Load Balancer

The internal Application Load Balancer is named buildndeploy-private-alb.

Unlike the public ALB, this load balancer is internal-facing and is used to route backend requests within the private network.

The internal ALB listens on the configured application port and forwards requests to the backend target group.

Backend Routing

Nginx receives frontend API requests beginning with /api/ and proxies them to the internal ALB. The internal ALB then forwards the requests to healthy backend instances.

Traffic Flow

Browser → Public ALB → Frontend EC2/Nginx → Internal ALB → Backend EC2

This approach allows the browser to use the public application endpoint without directly exposing the internal ALB.

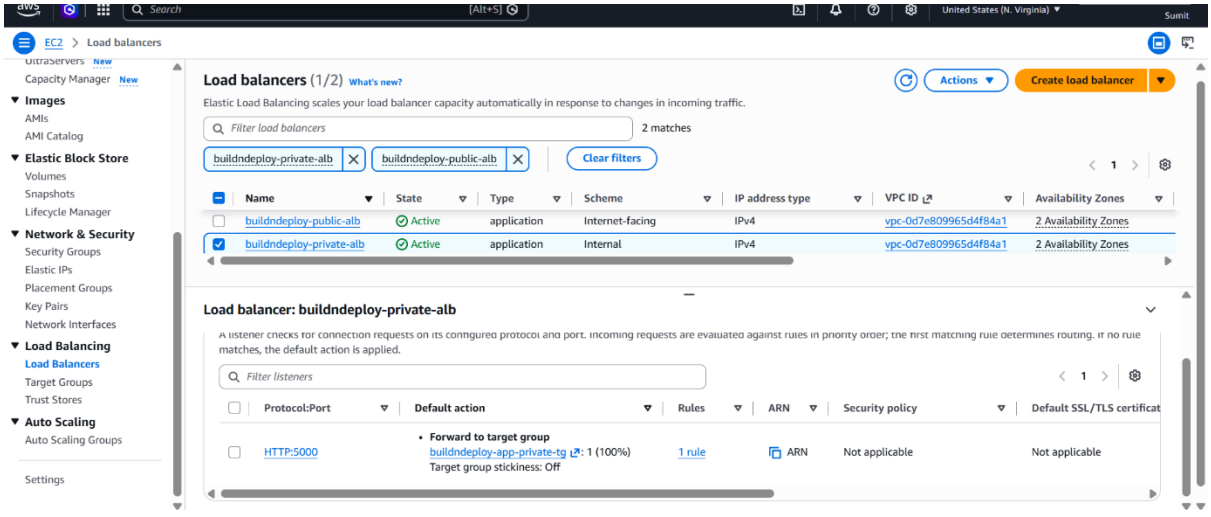


Figure 12: Internal Application Load Balancer configuration.

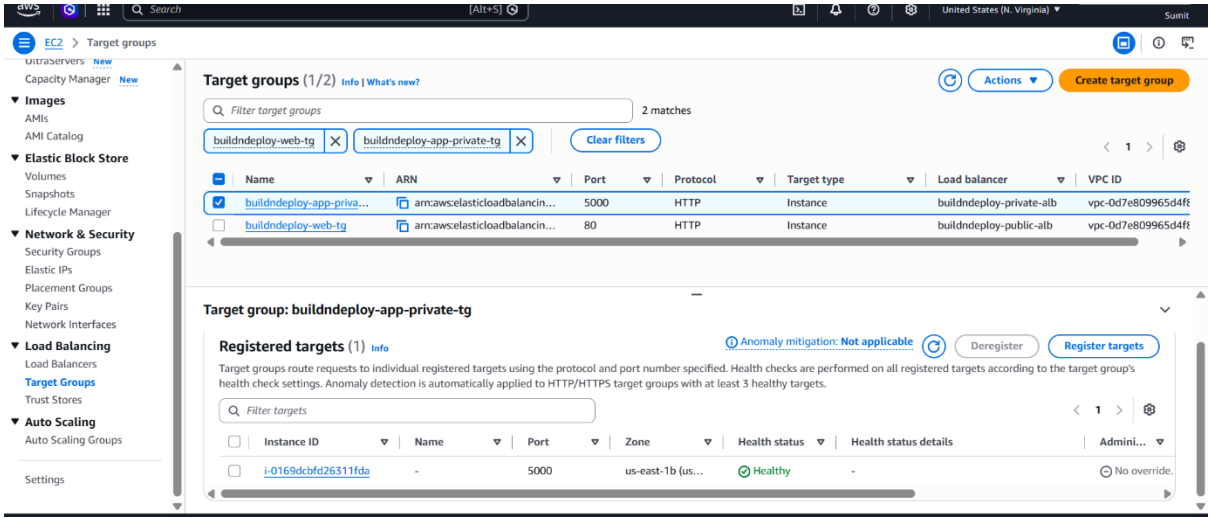


Figure 13: Backend target group, application port, and health check configuration.

7. Backend Layer

The backend runs on EC2 instances managed by buildndeploy-app-asg.

The application uses Node.js and Express.js to process API requests, perform application logic, and interact with the MySQL database.

The backend runs inside a Docker container and receives requests through the internal ALB.

Backend Components

- Node.js runtime.

- Express.js API application.
- Docker container.
- Amazon EC2.
- buildndeploy-app-asg Auto Scaling Group.
- buildndeploy-app-template Launch Template.
- buildndeploy-app-private-tg backend target group.

The backend exposes the following application endpoints:

Endpoint	Purpose
/api/health	Health check endpoint
/api/notes	Notes API endpoint supporting CRUD operations

CRUD refers to Create, Read, Update, and Delete operations.

The /api/health endpoint is used to check whether the backend application is responding as expected.

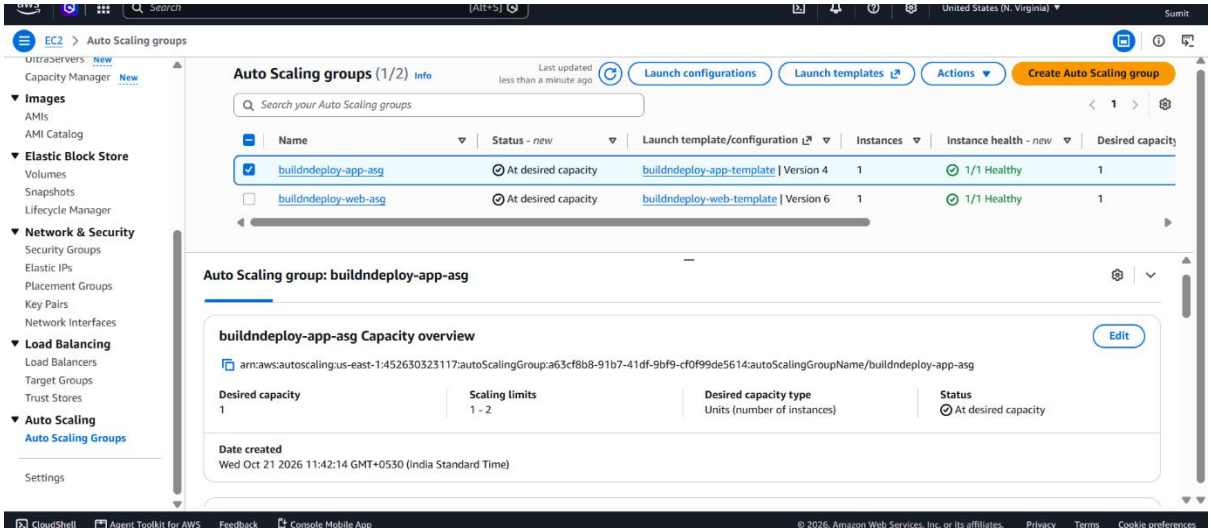


Figure 14: Backend Auto Scaling Group configuration.

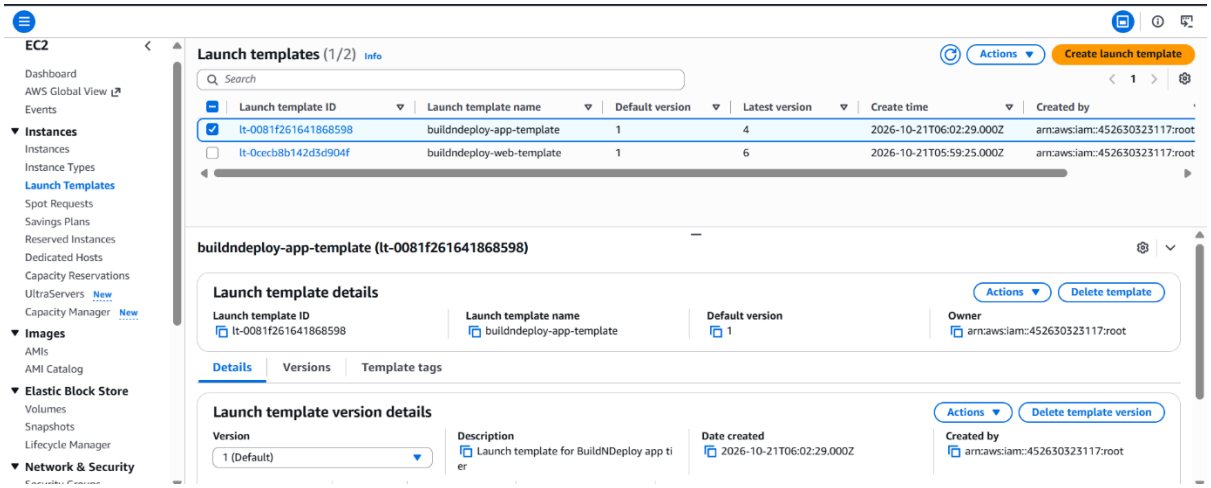


Figure 15: Launch Template used for backend EC2 instances.

8. Amazon EC2 Auto Scaling

Separate Auto Scaling Groups were used for the frontend and backend layers.

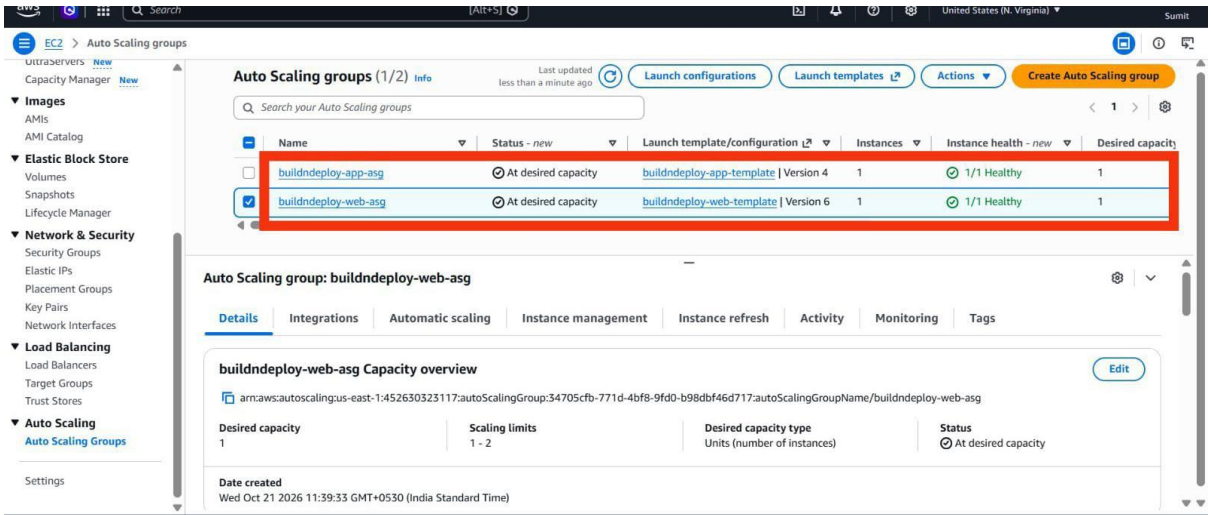
- buildndeploy-web-asg
- buildndeploy-app-asg

Each Auto Scaling Group uses its corresponding Launch Template to define the configuration for newly launched EC2 instances.

Auto Scaling can maintain the configured instance capacity and replace instances according to its health-check and scaling configuration.

During application updates, the configured instance refresh process replaces existing instances so that the updated Launch Template configuration can be applied.

Auto Scaling does not guarantee successful deployment by itself. The launch configuration, container startup, target health checks, available capacity, and refresh settings must all work correctly.



Sumit Milind Bhamkar || LinkedIn || GitHub

Figure 16: Separate Auto Scaling Groups for the frontend and backend.

9. Docker and Amazon ECR

Docker packages the frontend and backend applications into separate container images.

Amazon Elastic Container Registry (ECR) stores these images so they can be retrieved during deployment.

The project uses the following ECR repositories:

ECR Repository	Purpose
buildndeploy-frontend	Stores the frontend Docker image
buildndeploy-backend	Stores the backend Docker image

The EC2 instances pull the appropriate images from ECR and run the containers using the configured instance initialization and deployment process.

Docker provides application packaging, while ECR provides image storage. EC2 and Docker run the actual containers.

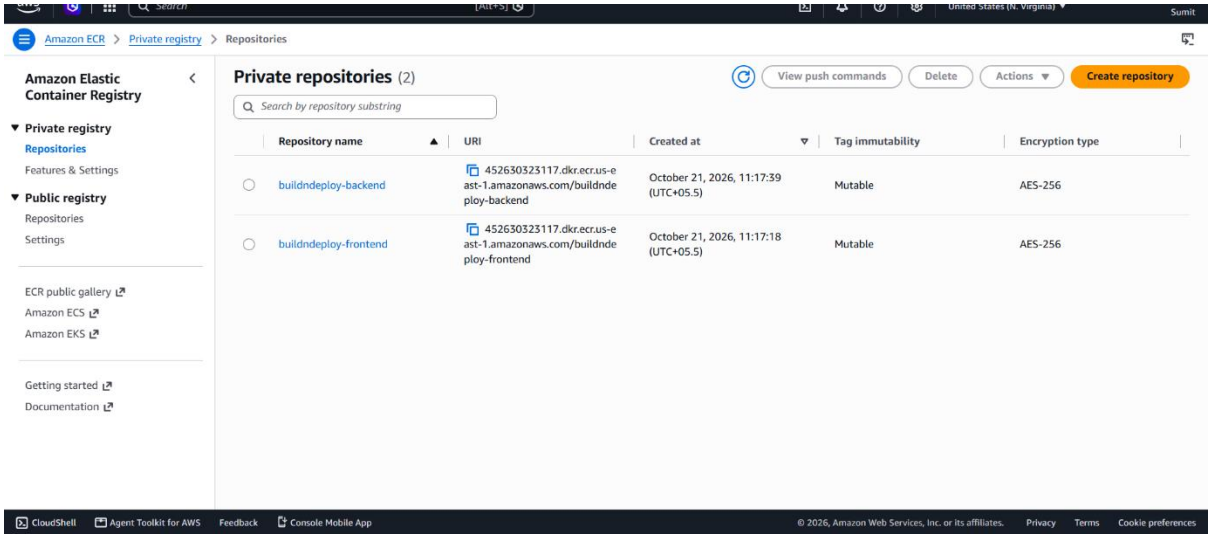


Figure 17: Frontend and backend container image repositories.

10. Database Layer — Amazon RDS for MySQL

Amazon RDS for MySQL provides managed relational database hosting for application data.

The backend connects to the database over private network connectivity using the configured database connection parameters.

The database security group restricts access to the authorized backend layer.

Database Connection Flow

Backend EC2 → Private VPC Network → RDS Endpoint → MySQL Database

The application uses database connection settings such as the database host, port, database name, and credentials. These values are provided through configuration rather than hard-coded into public source code.

The database was configured as a single-AZ deployment. The DB subnet group included subnets across two Availability Zones, but this alone does not create a Multi-AZ database deployment.

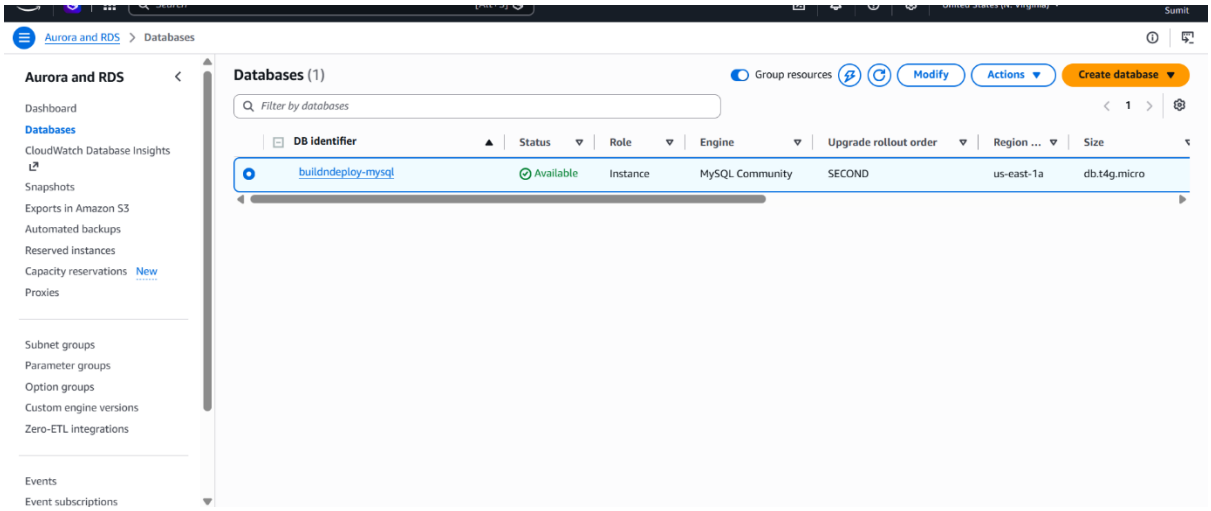


Figure 18: RDS for MySQL configuration.

11. AWS Systems Manager Parameter Store

AWS Systems Manager Parameter Store was used to manage application configuration and sensitive database parameters.

The project used the parameter path:

/buildndeploy/prod/

The parameters covered database connection settings such as the host, port, username, database name, and password.

The backend instances retrieve the required values using authorized AWS permissions.

Sensitive values must not be included in public documentation, screenshots, source code, or application logs. SecureString parameters should be used for sensitive values that require encryption at rest, with appropriate access permissions.

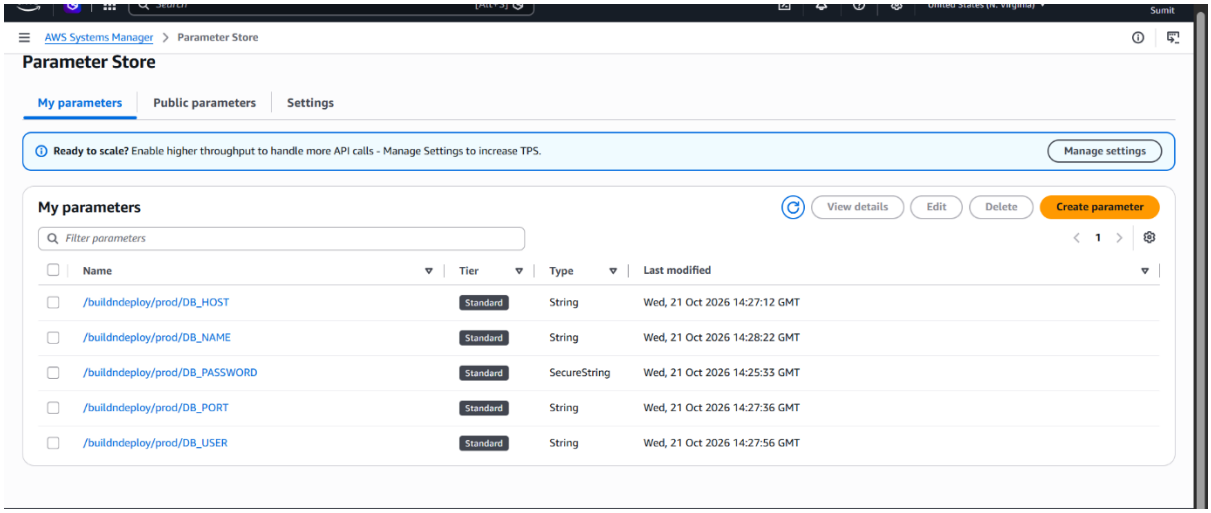


Figure 19: Parameter Store organization under the BuildNDeploy parameter path. Show parameter names only; never expose parameter values.

12. IAM and Security

AWS IAM controls which identities and services can access AWS resources.

BuildNDeploy uses IAM permissions for operations such as retrieving ECR images, accessing Parameter Store, and allowing the CI/CD workflow to update deployment resources.

The project also used GitHub OIDC for GitHub Actions authentication to AWS, avoiding the need to store long-lived AWS access keys in GitHub Actions secrets for that workflow.

The main security principles applied were:

- Separate public and private network layers.
- Restrict inbound access through Security Groups.
- Allow database access only from the required application layer.
- Use IAM roles for AWS service access.
- Limit permissions to required operations.
- Keep credentials and sensitive configuration out of public source code.
- Use OIDC for the configured GitHub Actions-to-AWS authentication flow.

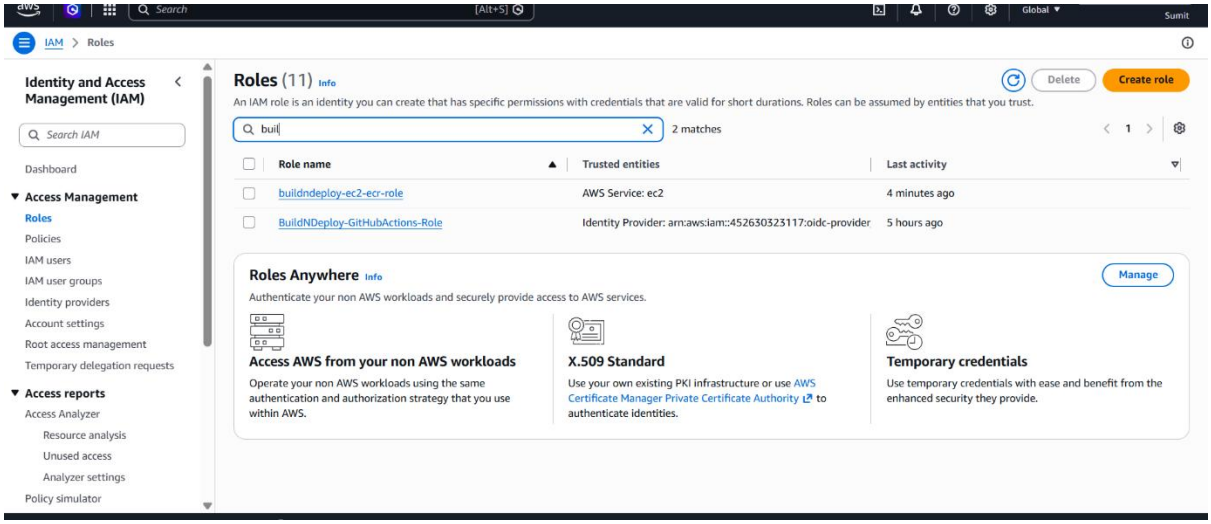


Figure 20: IAM and OIDC configuration used for deployment. Redact account IDs, role ARNs, and any sensitive identifiers before publication.

13. Domain and DNS — Amazon Route 53

Amazon Route 53 was used to manage DNS records for the project domain, buildndeploy.online.

The domain registration was handled through an external domain provider, while Route 53 handled DNS management.

The intended traffic path is:

User Browser → Domain DNS Resolution → buildndeploy-public-alb → Frontend Application

DNS resolution directs the user's browser to the public ALB. The load balancer then forwards HTTP traffic to the frontend target group.

The project was configured for HTTP and did not include HTTPS or AWS Certificate Manager configuration.

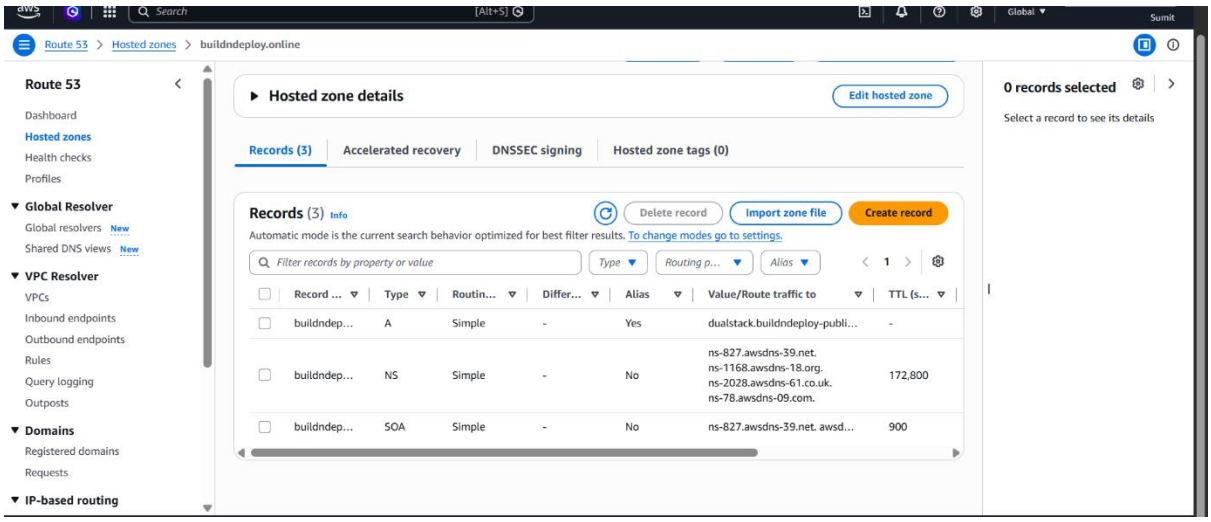


Figure 21: DNS records used to direct the application domain to the public entry point. Review the screenshot before publishing.

14. Application Request Flow — End to End

The following steps describe how the different layers work together.

1. A user opens buildndeploy.online in a browser.
2. DNS resolution directs the request to the public Application Load Balancer.
3. buildndeploy-public-alb forwards the request to a healthy target in buildndeploy-web-tg.
4. The frontend EC2 instance runs the Nginx container, which serves the React application.
5. When the user performs an operation requiring application data, the frontend sends an API request to the /api/ path.
6. Nginx proxies the API request to buildndeploy-private-alb according to its configuration.
7. The internal ALB forwards the request to a healthy backend instance in buildndeploy-app-private-tg.
8. The Node.js and Express.js application processes the request.
9. If database access is required, the backend connects to Amazon RDS for MySQL.
10. The backend returns the API response through the configured proxy path to the browser.

This flow separates external access, frontend delivery, backend processing, and database operations.

15. Monitoring and Health Checks

Application Load Balancer health checks are used to determine whether registered targets are responding as expected.

For the backend, /api/health was used as the health check path.

Healthy targets can receive traffic, while unhealthy targets are identified by the load balancer according to its configured health-check settings.

Amazon CloudWatch provides AWS monitoring capabilities for metrics, logs, and alarms. The available monitoring depends on which CloudWatch components and integrations are configured.

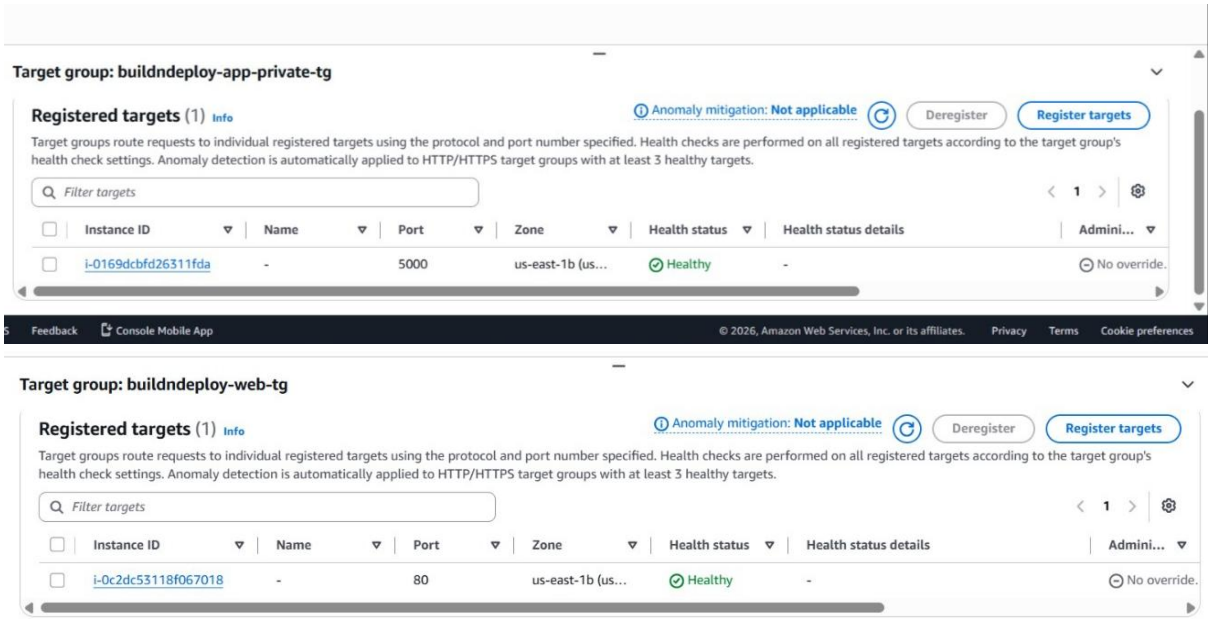


Figure 22: Application target health validation.

15.2 CloudWatch CPU Utilization Alarm

Amazon CloudWatch was used to monitor EC2 CPU utilization and detect high CPU usage.

A CloudWatch alarm named EC2-High-CPU-Alarm was configured with a threshold of **70% CPU utilization**. When CPU utilization reached or exceeded this threshold according to the configured evaluation settings, the alarm could enter the ALARM state.

Amazon SNS topic ec2-cpu-alerts was configured to send email notifications when the alarm triggered. An email notification was successfully received during testing.

This setup demonstrated how AWS monitoring can help identify potential performance issues and notify administrators when resource utilization becomes high.

Monitoring Flow:

EC2 Instance → CloudWatch CPU Metrics → CloudWatch Alarm (≥ 70%) → SNS Notification → Email Alert

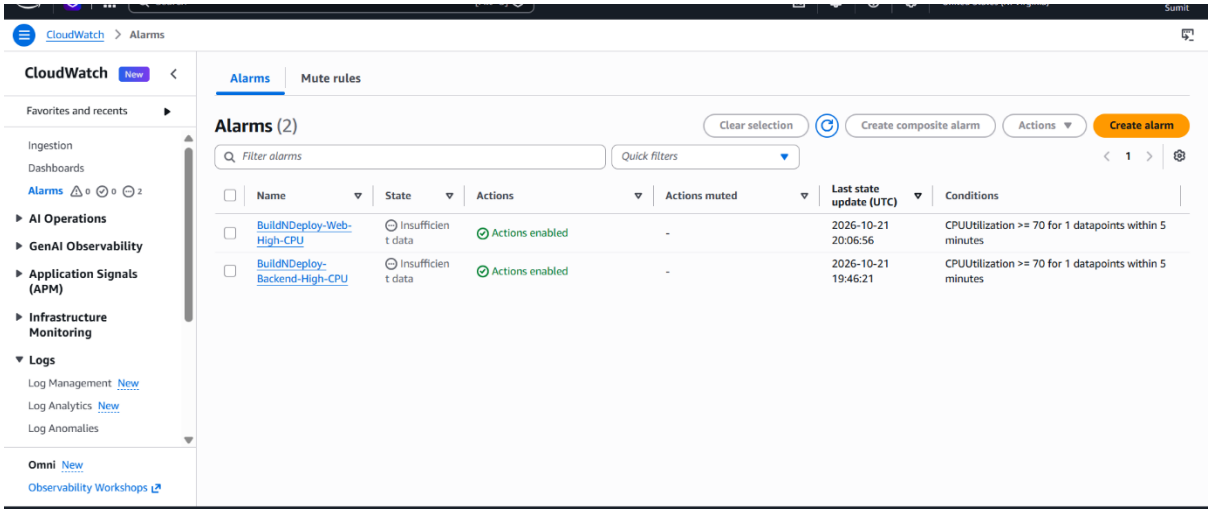


Figure 23: CloudWatch alarm configured to monitor EC2 CPU utilization with a 70% threshold.

16. CI/CD Pipeline Using GitHub Actions

The CI/CD pipeline automates the process of building application images and deploying updates to the AWS infrastructure.

The workflow starts when a developer pushes code to the configured GitHub repository and branch. GitHub Actions builds the required Docker images and publishes them to Amazon ECR.

The workflow authenticates to AWS using GitHub OIDC and an IAM role. It then updates the relevant Launch Template version and initiates an Auto Scaling instance refresh so that new instances can use the updated application configuration.

16.1 CI/CD Workflow

Developer → Git Push → GitHub Actions → AWS OIDC Authentication → Docker Build → Amazon ECR → Launch Template Update → Auto Scaling Instance Refresh → Application Update

16.2 Pipeline Steps

Step 1: Code Change

The developer modifies the frontend or backend application code and pushes the changes to the configured GitHub repository.

Step 2: Workflow Trigger

GitHub Actions starts the configured workflow when its trigger conditions are met.

Step 3: AWS Authentication

The workflow obtains AWS credentials through the configured GitHub OIDC trust relationship and IAM role, rather than relying on long-lived AWS access keys for this authentication flow.

Step 4: Docker Image Build

The workflow builds the relevant frontend or backend Docker image using the corresponding Dockerfile.

Step 5: Push Images to Amazon ECR

The workflow authenticates to ECR and publishes the new image to the appropriate repository:

- buildndeploy-frontend
- buildndeploy-backend

Step 6: Launch Template Update

The workflow updates the relevant Launch Template version with the required deployment configuration, including the updated image reference or other configured instance settings.

Step 7: Auto Scaling Instance Refresh

The corresponding Auto Scaling Group begins an instance refresh. New instances launch using the updated configuration and initialize the application containers.

Step 8: Health Validation

The application targets register with their respective target groups and must pass the configured health checks before being considered healthy.

Step 9: Application Verification

The deployment is validated by checking the GitHub Actions result, target health, and application behavior.

The actual outcome depends on the workflow configuration, permissions, image-tag strategy, instance bootstrap scripts, and Auto Scaling refresh settings.

16.3 CI/CD Workflow

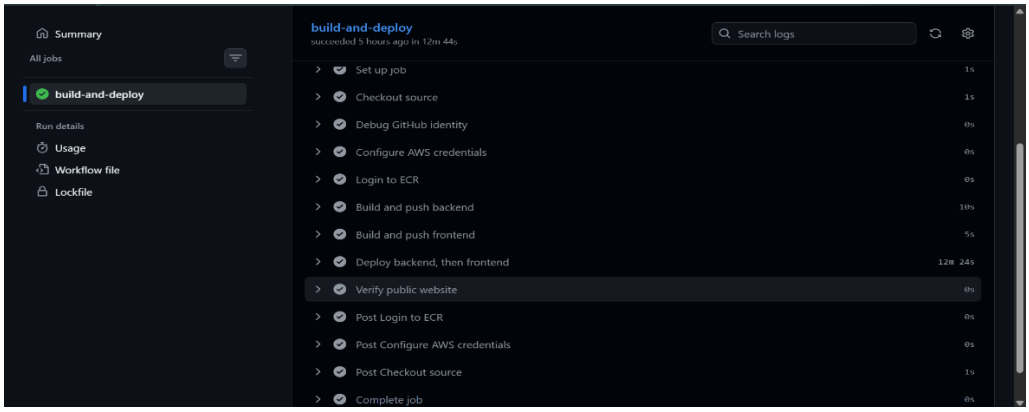
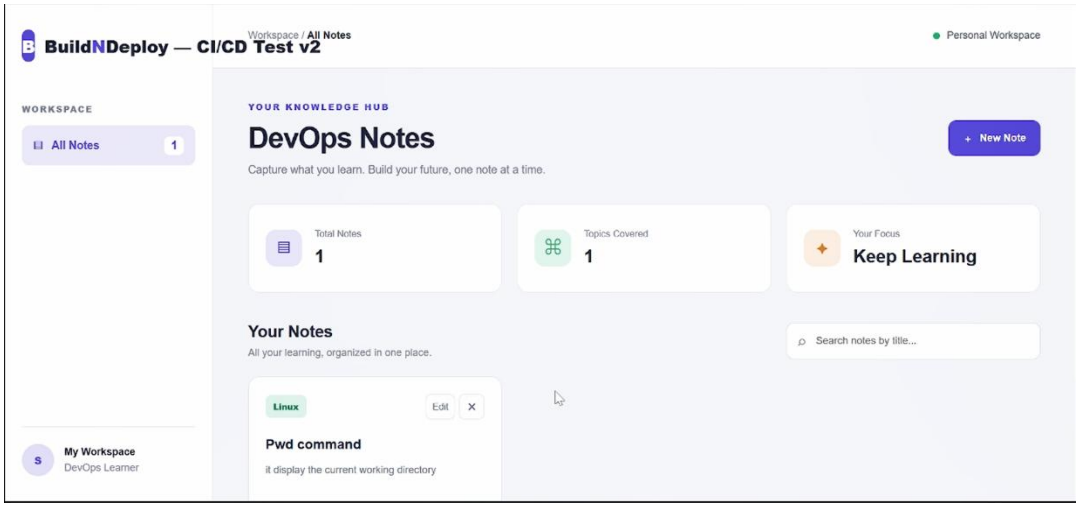


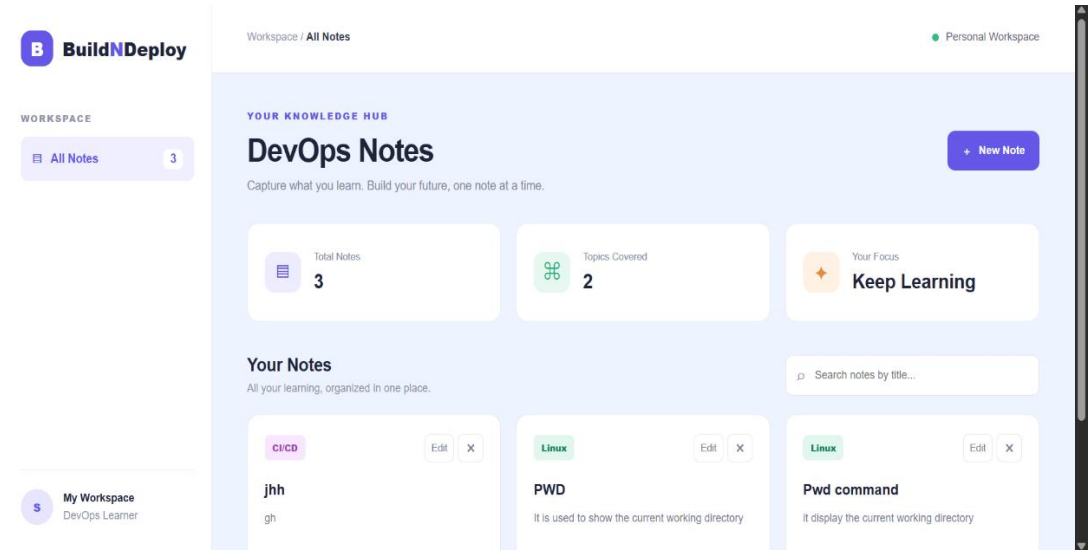
Figure 24: GitHub Actions workflow used to automate deployment.

For testing purposes, we modified the website's main heading and background to verify that the CI/CD pipeline successfully deploys code changes and reflects them on the live website.

Application Before:



Application After :



Sumit Milind Bhamkar || LinkedIn || GitHub

Figure 25: Application update demonstrated through the automated deployment workflow.

16.4 CI/CD Outcome

The pipeline demonstrates how application changes can be built, published, and deployed through an automated workflow rather than manually updating each application server.

It connects source control, Docker image creation, Amazon ECR, IAM-based authentication, Launch Templates, and Auto Scaling into one deployment process.

This approach helps make application updates repeatable and provides a clear workflow for validating changes after deployment.

17. Project Challenges and Learning Outcomes

During the project, several practical AWS and DevOps concepts were explored.

- Designing a VPC with public and private subnets.
- Routing external HTTP traffic through a public Application Load Balancer.
- Routing backend API requests through an internal Application Load Balancer.
- Configuring target groups and application health checks.
- Running Docker containers on EC2 instances.
- Publishing and retrieving container images through Amazon ECR.
- Managing EC2 capacity through Auto Scaling Groups.
- Using Launch Template versions during application updates.
- Configuring database connectivity between private application instances and RDS.
- Managing application configuration through Parameter Store.
- Configuring GitHub Actions authentication with AWS OIDC.
- Troubleshooting unhealthy targets, instance initialization, and deployment failures.
- Understanding the relationship between infrastructure configuration and successful application deployment.

These exercises provided hands-on experience with the way cloud networking, compute, containerization, database services, and CI/CD interact in an AWS environment.

18. Project Status and Limitations

BuildNDeploy was implemented as a hands-on learning project, and the AWS resources were subsequently cleaned up to prevent ongoing charges.

The documentation describes the architecture and workflow used during the project. It should not be interpreted as evidence that the AWS infrastructure is currently running.

The project was configured for HTTP rather than HTTPS, and the RDS database used a single-AZ deployment. These are important limitations to consider when evaluating the architecture for production use.

Potential future improvements include HTTPS configuration, stronger deployment rollback procedures, database backup and recovery testing, Multi-AZ database deployment, and expanded monitoring and alerting.

19. Conclusion

BuildNDeploy demonstrates the deployment of a containerized three-tier web application on AWS using a structured network architecture and automated CI/CD.

The project brings together Amazon VPC, EC2, Application Load Balancers, Auto Scaling Groups, Docker, ECR, RDS for MySQL, Parameter Store, IAM, Route 53, and GitHub Actions.

The architecture separates the frontend, backend, and database layers, while the CI/CD workflow automates image publishing and application instance updates.

This project provided practical experience in AWS infrastructure design, application routing, container deployment, access control, scaling, health checks, and deployment troubleshooting.

It serves as a foundation for further learning in AWS, DevOps automation, and container orchestration.

20. Key Enhancements and Future Improvements

The project can be improved further with the following enhancements:

- **HTTPS:** Secure website traffic using AWS Certificate Manager (ACM).
- **High Availability:** Configure Multi-AZ RDS for better database availability.
- **Monitoring:** Add CloudWatch dashboards, logs, and alarms.
- **Security:** Implement AWS WAF and strengthen IAM permissions.
- **CI/CD:** Add automated testing, image scanning, and rollback.
- **Infrastructure as Code:** Use Terraform to automate infrastructure creation.
- **Cost Optimization:** Monitor AWS usage and remove unused resources.

These are future improvements and were not implemented as part of the current project.

Sumit Milind Bhamkar || LinkedIn || GitHub